



SJÄLVSTÄNDIGA ARBETEN I MATEMATIK

MATEMATISKA INSTITUTIONEN, STOCKHOLMS UNIVERSITET

Gröbner Bases and Elimination in Macaulay 2

av

Christian Eriksson

2026 - No 5

Gröbner Bases and Elimination in Macaulay 2

Christian Eriksson

Självständigt arbete i matematik 15 högskolepoäng, grundnivå

Handledare: Sofia Tirabassi

2026

Abstract

This thesis covers fundamental concepts in affine spaces and varieties, polynomial elimination and Gröbner bases. It accomplishes this by introducing affine spaces and varieties to allow a better understanding of what algebraic geometry is, and then pivots to exploring Gröbner bases. This is done with the final goal of constructing an algorithm that can compute these bases deterministically. First, monomial ideals are considered and then the scope is extended to polynomial ideals. A verification of membership to Gröbner bases, the Buchberger criterion, is proven. The Buchberger algorithm, or the construction of Gröbner basis, is implemented in the computer algebra system Macaulay2 using the criterion. Univariate division, multivariate division, and solutions to systems of polynomial equations were also implemented algorithmically. The algorithms were verified programmatically with unit tests. The thesis provides an interesting introduction to the construction and implementation of algorithms as well as a solid foundation for future expansion into algebraic geometry or algorithms construction.

Contents

1	Introduction	9
2	The Algebraic Abstraction of Geometry	10
2.1	Polynomials	10
2.2	Affine Spaces and Varieties	11
2.2.1	Affine Spaces	11
2.2.2	Polynomials in Affine Spaces	13
2.2.3	Affine Varieties	13
3	Algebra Theory	15
3.1	Rings and Ideals	15
3.1.1	Monomial Ideals	15
3.1.2	Dicksons Lemma	16
3.2	Hilbert Bases Theorem	17
3.3	Gröbner Basis	18
3.3.1	Properties Of Gröbner Basis	19
3.3.2	Buchberger's Criterion	19
4	Algorithms in Macaulay2	23
4.1	Exploring Macaulay2	23
4.2	Division Algorithms	23
4.2.1	Univariate Division Algorithm	24
4.2.2	Multivariate Division Algorithm	24
4.3	The Buchberger Algorithm	26
4.4	Solving Systems of Polynomials	28
4.4.1	Book Examples of Polynomial Solutions	28
4.4.2	Solving a System of Polynomial Equations	28
A	Macaulay2 source files	31
A.1	Algorithm121.m2	31
A.2	Algorithm123.m2	31
A.3	Algorithm124.m2	34
A.4	Algorithm161.m2	36
A.5	BookExample1612.m2	39
A.6	BookExample1613.m2	40

A.7 BookExample2.1.1.m2	41
A.8 BookExample2.2.m2	42
A.9 BookExample222.m2	42
A.10 BookExample231.m2	42
A.11 BookExample232.m2	42
A.12 DocumentationConditionals.m2	43
References	45

1 Introduction

Something goes here

2 The Algebraic Abstraction of Geometry

Algebra can be seen as the mathematical expression of the human need for structure and abstraction. Through algebra, humans try to capture more of the mathematical, and natural, domain under their pen. The endless hunger for knowledge can express itself in various way. When using algebra as a tool, creating abstract models that capture generic structure is favored. However, these abstract models can be difficult to understand. The ability to navigate between levels of abstraction is a vital tool for any mathematician interested in applying algebraic methods.

This paper assumes a degree of familiarity with algebraic structures, but the uninitiated peruser is kindly directed towards the digestable "Introduction to Modern Algebra" [Wei70] with guiding exercises, or alternatively, the more rigorous "Abstract Algebra" [DF03] for a deeper understanding.

As the section header suggests, this section will be primarily focused with defining the tools necessary to build an analytic bridge between geometry and algebra so that we can analyze geometric objects through algebra. A function that maps geometric objects to algebraic must be established. This function uses Affine Varieties and Ideals, along with the dictionary established in the Hilbert Strong Nullstellensatz, to accomplish that goal.

2.1 Polynomials

In order to discuss mathematical concepts, a compact object is required. This is done by compressing information into a single mathematical expression. In order to discuss polynomials rings, we begin with defining least compressed case, monomials. Then, we consider a collection of monomials, a polynomial. Finally, we can consider the compact object, polynomial rings.

Definition 2.1. A **monomial** in $x_1, \dots, x_n$ is a product of the form

$$x^\alpha = x_1^{\alpha_1} \cdot x_2^{\alpha_2} \cdots x_n^{\alpha_n}$$

where all $\alpha = \alpha_1, \dots, \alpha_n$ are nonnegative integers.

As the root prefix "mono" suggests, there is only one of these terms x^α . They are independent and self-contained. However, as we are creating the tools to generalize and deal with more abstract concepts, we also want to include the ability to capture any linear combination of these monomials.

Definition 2.2. A **polynomial** f in $x_1, \dots, x_n$ with coefficients, $a_1, \dots, a_n$, in a field k is a finite linear combination of monomials.

$$f = \sum_{\alpha} a_{\alpha} x^{\alpha}, \quad a_{\alpha} \in k$$

With the polynomial definitions at hand, we can construct the final abstraction for our mathematics involving polynomials, the polynomial ring. This is defined as $k[x_1, \dots, x_n]$. When leveraging this powerful abstraction, we capture n dimensions of variables on any field k . This achieves the goal of compact analysis.

2.2 Affine Spaces and Varieties

For the purpose of this thesis, geometry will be restricted to how points, lines, planes (and higher dimensional objects) interact with space. To build a bridge between algebra and geometry, there must be an established way to discuss geometry algebraically. This is where the concepts of affine spaces comes in.

2.2.1 Affine Spaces

Let us take a moment to reflect around how one might represent a geometric object. Consider a line. One could use the well known two dimensional cartesian plane, plot the points $(x_1, y_1), (x_2, y_2)$ and then pull a pen between the two points. Here, a line as been created, but it is dependent on the following: a coordinate system and a restriction of dimension so we can physically represent it. These restrictions are undesirable when studying a line algebraically, so we must circumvent these restrictions. The solution is affine spaces. Affine spaces are coordinate-agnostic spaces. This eliminates our need for the coordinate system that was a prerequisite when drawing the lines, which is a good start. The affine space is also easily extended beyond the physical.

Definition 2.3. An *affine space* is either the empty set, or a triple $\langle E, \vec{E}, + \rangle$. E is a nonempty set of points while $\vec{E}$ is a vector space (of translations). The action $+$ is an operation on both E and $\vec{E}$ defined as follows: $+: E \times \vec{E} \rightarrow E$. The following axioms are assumed for affine space:

1. $a + 0 = a$ for every $a \in E$.
2. $(a + u) + v = a + (u + v)$ for every $a \in E$ and every $u, v \in \vec{E}$.

3. For any two points $a, b \in E$, there is a unique $u \in \vec{E}$ such that $a + u = b$

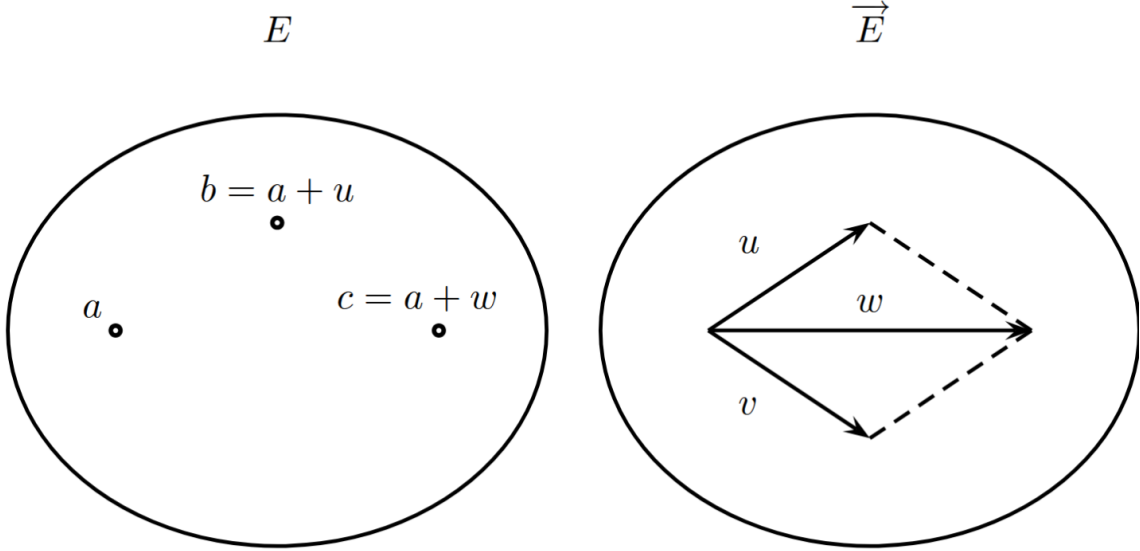


Figure 1: Intuitive understanding of affine space [Gal25]

A quick note of importance on each axiom. If you start at point a and do nothing, you should still be at a . Axiom two deals with the associative property. The ability to rearrange calculations simplifies any algebra involved. The last axiom, that of uniqueness is computationally useful. Assuming that an unique element in $\vec{E}$ always exists that can connect two points a and b is useful when solving these problems on a higher level of abstraction, especially when a computer that cannot handle errors is involved.

The intuition behind the coordinate-agnostic nature should be clear when considering figure 1. Here a separation between the vectors and the points *to which the vector is applied to* should be noted. These two can be defined independently and therefore there is no system tying them together. An example is provided for further clarity.

Example 2.4. Given a field k and a positive integer n , define the n -dimensional *affine space* over k as:

$$k^n := \langle \{[\hat{x}_1, \dots, \hat{x}_n]^T : \hat{x}_i \in k, i = 1, \dots, n\}, \vec{k}^n, + \rangle$$

Where $\vec{k}^n$ is the vector space of translations, and $+$ denotes the action of shifting a point by a vector via component-wise addition.

To further our understanding of the space, we can use the concrete example where the field in question is $\mathbb{R}$. Then, we get the familiar space $\mathbb{R}^n$. So the affine space is more abstract than the one we are used to dealing with from real world applications where we are restricted.

2.2.2 Polynomials in Affine Spaces

With a better understanding of affine spaces, we will consider how polynomials relate to this space. Using the definition 2.2 we define the polynomial *function* $p \in k[x_1, \dots, x_n]$ as follows:

$$p : k^n \rightarrow k$$

The function maps the multidimensional field onto the single dimension. This p takes every $x_1, \dots, x_n$ and replaces it with $a_1, \dots, a_n \in k[x_1, \dots, x_n]$. This is done easily by substituting every $x_i = a_i$ in the polynomial p . Since all the substituted a 's are elements $k[x_1, \dots, x_n]$, then $p(a_1, \dots, a_n)$ is also in $k[x_1, \dots, x_n]$.

2.2.3 Affine Varieties

With an understanding of how the space we will operate upon will work and how the polynomials can be evaluated on that space, the first algebraic structure of geometry will be discussed: affine varieties.

Definition 2.5. Let k be a field, and let $f_1, \dots, f_s$ be polynomials in $k[x_1, \dots, x_n]$. Then, set

$$\mathbf{V}(f_1, \dots, f_s) = \{(a_1, \dots, a_n) \in k^n \mid f_i(a_1, \dots, a_n) = 0 \ \forall 1 \leq i \leq s\}$$

We call $\mathbf{V}(f_1, \dots, f_s)$ the **affine variety** defined by $f_1, \dots, f_s$.

In other words, the affine variety is the set of $(a_1, \dots, a_n)$ that solve all the equations defined in the variety simultaneously. Or, it is the intersection of all the polynomial roots. Even though $\mathbf{V}$ might simply look like a graphed equation, figure 2a only provides the *base* for intuition. While it may look like a regular circular graph, with familiar x and y coordinates, the keen observer will note that we are dealing with two dimensions, x_1, x_2 and that the graph is in fact the where this polynomial studied is equal to 0. If we extract the polynomial from the variety and set it to 0, we end up with $x_1^2 + x_2^2 = 1$. Figures 2b and 2c show the increasing

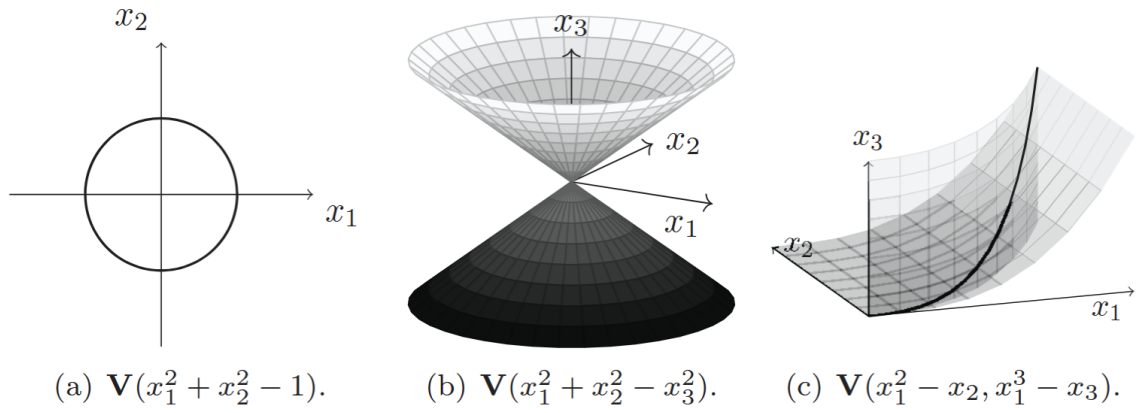


Figure 2: Examples of affine spaces [LM21]

complexity in three dimensions. Graphing this equation makes the roots of the polynomial self-evident, but graphing it is neither necessary or desirable. It is only a tool to inform intuition. This is especially true as we are interested in n dimensions and graphing will become next to impossible.

3 Algebra Theory

With affine varieties under our belt, there is an obvious follow up to this knowledge, especially for those interested in the real application of this mathematics. Namely, how do we solve these varieties? Luckily, the problem of asking for the affine variety points can be reduced to solving a system of polynomials equations. This is easier said than done, and computational complexity arises when considering n dimensions. Most real world problems will involve many dimensions, and being able to solve these problems quickly and with confidence is of importance. To do so, utilizing a computer algorithm would be a solution.

Every computer language has important data structures that determine how the language is shaped. For Macaulay 2, ideals is one of these types. They are important to algebra, and native support for manipulating ideals exist. However, to efficiently handle ideals, there is a problem that needs to be solved with great confidence. Namely: how do we go between abstract entities such as ideals and something that a computer can interact with?

3.1 Rings and Ideals

The ideal description problem formulates the bridge between the abstract and machine world. It does this by concretizing the abstract concept of an ideal to a tangible set of generating polynomials that can be interacted with and evaluated. In order to bridge this gap, finite polynomial ideals have to be found. Starting by analyzing monomial ideals in $k[x_1, \dots, x_n]$ is the first step.

3.1.1 Monomial Ideals

Much like when introducing affine varieties, starting with a single monomial instead of the combination of them reduces complexity. So, when studying the ideals, we start by considering a less complex problem.

Definition 3.1. An ideal $I \subseteq k[x_1, \dots, x_n]$ is a monomial ideal that is determined by a set $A \subset \mathbb{Z}^n$. All polynomials in the ideal can be written as a linear combination of $\sum_{\alpha \in A} h_{\alpha} x^{\alpha}$ where h_{α} lies in $k[x_1, \dots, x_n]$.

Monomial ideals can largely be thought of as their exponents. This is their main benefit and why they are important when dealing with computer algebra systems.

We will see this property in the coming proofs. As an example, consider the following lemma:

Lemma 3.2. *Let $I = \langle x^\alpha | \alpha \in A \rangle$ be a monomial ideal. Then a monomial x^β lies in I if and only if x^β is divisible by x^α for some $\alpha \in A$.*

Proof. $\Leftarrow$ This can be proven by considering the exponents. Starting off with showing x^β to x^α , the exponents can be leveraged. If β is a multiple of α then x^β is in the ideal by definition since x^α generates the ideal.

$\Rightarrow$ Using the definition of the monomial ideal, we can write x^β as the polynomial sum of generators of the monomial ideal and then write h_i as the sum of generators again.

$$x^\beta = \sum_{i=1}^s h_i x^{\alpha(i)} = \sum_{i=1}^s \left(\sum_j c_{i,j} x^{\beta(i,j)} \right) x^{\alpha(i)} = \sum_{i,j} c_{i,j} x^{\beta(i,j)} x^{\alpha(i)}$$

Here we can see that every term on the right hand side is divisible by x^α , so then x^β on the left hand side must also be so. This proves that we can follow our intuition regarding the exponents and their divisibility. □

3.1.2 Dicksons Lemma

This lemma demonstrates the usefulness of monomial ideals. The ability to reduce ideals to finite generators is a step towards closing the gap between the machine and abstract.

Theorem 3.3. *Let $I = \langle x^\alpha | \alpha \in A \rangle$ be a monomial ideal. Then I can be finitely written by $I = \langle x^{\alpha(1)}, \dots, x^{\alpha(s)} \rangle$, where all α lie in A .*

Proof. Consider an inductive approach. Starting off with $n = 1$ for our $k[x_1, \dots, x_n]$, it is easy to see that the smallest element of $A \subseteq \mathbb{Z}_{\geq 0}$ generates the remaining elements. With $n > 1$, we start our induction. Consider the n th case. We have the monomial ideal, call it J , from the $n - 1$ case. This is finite by our induction, so when our A goes from a subset of $\mathbb{Z}_{\geq 0}^{n-1}$ to $\mathbb{Z}_{\geq 0}^n$, we can generate slices of these J ideals with the new $x_n = y$ variable. We take some sufficiently large $m \geq 0$ and group the J_l ideals by their corresponding y value up to $m - 1$. In other words, let the ideal J_l be generated by x^β where $x^\beta y^l \in I$. These J_l slices form a topological map of the ideal I . Thus, I is also finite. □

After a finite ideal is generated, the monomial ideal membership problem is solved. This is since we can take a polynomial and check if the remainder from division by the finite monomial ideal is 0. If it is, it belongs to the monomial ideal since one of the members will generate it.

3.2 Hilbert Bases Theorem

In this section, the result that monomial ideals have finite generators is extended to polynomial ideals. The basic idea is that by rearranging the ideal to a monomial ideal using the leading term, we can apply the Dicksons lemma. The first step towards bridging the gap between polynomial and monomial ideals is by crafting an ideal of interest. Namely, the ideal generated by the leading term of a polynomial ideal.

Definition 3.4. Let $I \subseteq k[x_1, \dots, x_n]$ be an ideal other than $\{0\}$ and fix a monomial ordering. Then

1. $LT(I) = \{cx^\alpha \mid \text{there exists } f \in I \setminus \{0\} \text{ with } LT(f) = cx^\alpha\}.$
2. $\langle LT(I) \rangle$ is the ideal generated by the elements of $LT(I)$.

$LT(I)$ is a large set, but it is the bridge between general ideals and monomial ideals. After all, the difference between a monomial and the lead term of the polynomial are the other monomials in the polynomial. If we can capture them by constructing an ideal that generates everything by using the leading term, we can create a superset to operate upon. $\langle LT(I) \rangle$ is that set.

Proposition 3.5. Let $I \subseteq k[x_1, \dots, x_n]$ be an ideal different from the $\{0\}$.

1. $\langle LT(I) \rangle$ is a monomial ideal.
2. There are $g_1, \dots, g_t \in I$ such that $\langle LT(I) \rangle = \langle LT(g_1), \dots, LT(g_t) \rangle$.

Proof. Since we have used the leading term to construct an ideal, the difference between the monomial ideal and $\langle LT(I) \rangle$ is a nonzero constant. Consider a single $g_i = g$, and break the lead term down to the lead coefficient and lead monomial. $LT(g) = LC(g) \times LM(g)$ means that $LT(g)$ is a field scalar of $LM(g)$, which means that $LT(g) \in \langle LM(g) \rangle$. We can multiply by the inverse to show that $LT(g) \times LC^{-1}(g) = LM(g)$. But $LC^{-1}(g)$ is still in $k[x_1, \dots, x_n]$, so the same logic applied and they are equal to each other. So, $\langle LT(I) \rangle = \langle LM(I) \rangle$. The same logic is used

for the second item. Using Dicksons Lemma tells us that monomial ideals are finitely generated. Since $\langle LT(I) \rangle$ only differs from a monomial ideal by a nonzero constant, Dicksons lemma can be applied here as well and I is finitely generated. $\square$

With all this information accumulated, we can formally prove that every polynomial ideal has a finite generating set. This theorem is known as the Hilbert Basis Theorem.

Theorem 3.6 (Hilbert Bases Theorem). *Every ideal $I \in k[x_1, \dots, x_n]$ has a finite generating set. In other words, $I = \langle g_1, \dots, g_t \rangle$ for some $g_1, \dots, g_t \in I$.*

This proof is outside of the scope, but is given in chapter 2 of Cox [DAC10] as Theorem 4 on page 77.

3.3 Gröbner Basis

The generating set $\langle LT(I) \rangle$ was introduced in the last section. Dealing with monomials reduces the complexity drastically as opposed to polynomials, so if we can use the $\langle LT(I) \rangle$ instead of the polynomial one, it would simplify calculations. Recall that the Hilbert Basis theorem stated that every polynomial ideal has a finite generating set. However, it is not unique. We will start with an ideal that is of special interest to the world of polynomial calculation.

Definition 3.7. Choose a monomial ordering on the polynomial ring $k[x_1, \dots, x_n]$. A finite nonzero subset, $G = \{g_1, \dots, g_t\}$ of an ideal, I , which is a subset of $polyRing$ is a **Gröbner basis** if

$$\langle LT(g_1), \dots, LT(g_t) \rangle = \langle LT(I) \rangle \quad (1)$$

The construction is purposeful; we can operate on a object of immense size, $\langle LT(I) \rangle$, while restricting ourselves to a finite number of generators. This makes the theoretical exploration less complex due to the flexibility of $\langle LT(I) \rangle$. It also ensures that giving instruction to a computer becomes possible as we have eliminated the need to find an infinite ideal. Instead, we find the finite generating set.

The next question to answer is how to construct Gröbner bases to operate upon. Since these are useful for computational purposes, an algorithm for how to compute them will be discussed and proven. Before that, it is important to have an understanding of some key properties we can leverage from this basis.

3.3.1 Properties Of Gröbner Basis

The order of the dividing polynomials affects the outcome when performing multivariate division. This is problematic when establishing deterministic outcomes. By fixing a lexicographical order, Gröbner bases guarantee a unique remainder.

Proposition 3.8. *Let $I \subseteq k[x_1, \dots, x_n]$ be an ideal and let $G = \{g_1, \dots, g_t\}$ be a Gröbner basis for I . Then given $f \in k[x_1, \dots, x_n]$, there is a unique $r \in k[x_1, \dots, x_n]$ with the two following properties:*

1. *No term of r is divisible by any of $LT(g_1), \dots, LT(g_t)$.*
2. *There is a $g \in I$ such that $f = g + r$.*

Proof. Consider the division algorithm $f = q_1g_1 \cdots + q_tg_t + r$. Every $LT(g_t)$ is already divided out. In other words, if r had been divisible by the lead term there would have been a corresponding q_t to engulf it. That proves (1). (2) is solved by considering the fact that we are operating with Gröbner bases; we must leverage our ability to show that $\langle LT(I) \rangle = \langle LT(g_1), \dots, LT(g_t) \rangle$. To prove uniqueness, assume that the remainder is non-unique and use lemma 3.2 to disprove that assumption. In other words: assume that $f = g + r = g' + r'$ follows our proposition. Rearranging, $r - r' = g - g' \in I$. Now, using the special property of the basis, $LT(r - r') \in \langle LT(I) \rangle = \langle LT(g_1), \dots, LT(g_t) \rangle$. Since $LT(r - r')$ is on $LT(g_i)$, then it is also divisible by that same $LT(g_i)$. Using the first property, $r - r'$ cannot be divided by $LT(g_i)$, so in order for the division to succeed, $r - r' = 0$. This contradicts the assumption that r was non-unique. $\square$

Another important property is how easily one can verify if a function is in an ideal. The membership problem is solved by dividing f by G and checking if the remainder is zero. If it is, then the function belongs to the ideal. This membership check's concrete nature is useful for computational applications.

3.3.2 Buchberger's Criterion

The following definitions and lemmas are preparatory for algorithmically finding Gröbner bases. The algorithm and my implementation using Macaulay2 can be found in section 4.3.

Definition 3.9. We will write $\overline{f}^F$ for the remainder on division of f by the ordered s-tuple $F = (f_1, \dots, f_s)$. If F is a Gröbner basis for $\langle f_1, \dots, f_s \rangle$, then we can regard F as a set.

Now that we have a concise way to refer to the remainder of a division by two arbitrary parameters, our dialog around the subject will be less cluttered. A deterministic approach to cancelling leading terms would help in the construction of our algorithm. Below is a definition that introduces the tool we will use to craft the algorithm's kernel.

Definition 3.10. Let $f, g \in k[x_1, \dots, x_n]$ be nonzero polynomials.

1. If $\text{multideg}(f) = \alpha$ and $\text{multideg}(g) = \beta$, then let $\gamma = (\gamma_1, \dots, \gamma_n)$, where $\gamma_i = \max(\alpha_i, \beta_i)$ for each i . We call x^γ the least common multiple of $LM(f)$ and $LM(g)$, written $x^\gamma = \text{lcm}(LM(f), LM(g))$.
2. The S-polynomial of f and g is the combination

$$S(f, g) = \frac{x^\gamma}{LT(f)} \cdot f - \frac{x^\gamma}{LT(g)} \cdot g \quad (2)$$

Let us explore the definition provided by the book [DAC10]. Our numerators share the extracted least common multiple. The other interesting part is that the incoming parameterized functions, f, g , are being divided by their own leading term. It seems as if though the least common multiple ensures that a cancellation occurs. In other words, this S-polynomial could be an engine for reducing the degree of polynomials. The next lemma showcases an exiting part of mathematics. Namely, validating intuition.

Lemma 3.11. *Suppose we have a sum $\sum_{i=1}^s p_i$ where $\text{multideg}(p_i) = \delta \in \mathbb{Z}_n^{\geq 0}$ for all i . If $\text{multdeg}(\sum_{i=1}^s p_i) < \delta$, then $\sum_{i=1}^s p_i$ is a linear combination with coefficients in k , of the S-polynomials $S(p_j, p_l)$ for $i \leq j, l \leq s$. Furthermore, each $S(p_j, p_l)$ has $\text{multidegree} < \delta$.*

Cox and his book [DAC10] turns our attention to an interesting idea. The assumption made in the lemma relies heavily on the properties of the s-polynomial. The written proof exists in the book in chapter 2.6 as Lemma 5. The intuition to extract from the proof can be described as follows.

Consider an addition of polynomials, all of them equal in multidegree, summing to a lesser multidegree. If you can find an example of that, this lemma ensures that you can rewrite that addition of polynomials as an addition of s-polynomials. Using this fact, the criteria for admission into the gröbner basis can be created.

Theorem 3.12 (Buchberger's Criterion). *Let I be a polynomial ideal. Then a basis $G = g_1, \dots, g_t$ of I is a Gröbner basis of I if and only if the remainder of the S-polynomial constructed using g_i, g_j ($i \neq j$) by G is zero. Or in other words if: $\overline{S(g_i, g_j)}^G = 0$ for all $(g_i, g_j) \in G$ where $(i \neq j)$.*

Here, the idea behind finding Gröbner bases is shown. All one has to do to find a Gröbner basis is to simply check all the g_t s against each other using the constructed S-polynomial. If a nonzero remainder is found, a new g_{t+1} is added to the basis being generated. If the remainder is zero, then we are positive that we have a Gröbner basis.

Proof. $\Rightarrow$ If G is a Gröbner basis, then we are trying to prove the remainder $\overline{S(g_i, g_j)}^G$ is 0. Leveraging the problem of membership, we can recall that the remainder would be 0 for all members of the Gröbner basis. Since all of the S-polynomials are on I , the remainder is 0.

$\Leftarrow$ We start by letting $f \in I$ be nonzero. Prove that

$$LT(f) \in \langle LT(g_1), \dots, LT(g_n) \rangle$$

This is done by considering the following functions:

$$f = \sum_{i=1}^t h_i g_i, h_i \in k[x_1, \dots, x_n] \quad (3)$$

$$\text{multdeg}(f) \leq \max \{ \text{multdeg}(h_i g_i) \mid h_i g_i \neq 0 \} \quad (4)$$

$$\delta = \max_{1 \leq i \leq t} \{ \text{multdeg}(h_i g_i) \} \quad (5)$$

By equations 4 and 5 we can arrive at the fact $\text{multdeg}(f) \leq \delta$. In addition to this, we will also minimize the δ . Since we have that $\text{multdeg}(f) \leq \delta = \min \text{multdeg}(f)$, there needs to be two separate cases.

1. $\text{multdeg}(f) = \delta$. Then, $LT(f)$ is divisible by $LT(g_i)$ since the gröbner base produces all leading terms in the ideal. In other words:

$$LT(f) \in \langle LT(g_1), \dots, LT(g_n) \rangle$$

2. $\text{multdeg}(f) < \delta$.

Proving $\text{multdeg}(f) < \delta$ requires rigor. Lets us first state an assumption which we will operate upon: The arrangment of f provides a minimal δ cannot be reduced further. If that assumption can be broken by finding a smaller δ , then we have proven that δ will always be the $\text{multdeg}(f) = \delta$ case.

We do this by considering an f with a minimum δ . Split the sum into its two components and then extract the leading term for each iteration of i .

$$\begin{aligned} f &= \sum_{\text{multdeg}(h_i g_i) = \delta} h_i g_i + \sum_{\text{multdeg}(h_i g_i) < \delta} h_i g_i \\ &= \sum_{\text{multdeg}(h_i g_i) = \delta} LT(h_i) g_i + \sum_{\text{multdeg}(h_i g_i) = \delta} (h_i - LT(h_i)) g_i + \sum_{\text{multdeg}(h_i g_i) < \delta} h_i g_i \end{aligned} \quad (6)$$

Clearly the sum of functions with multidegree less than δ cannot sum to something greater than δ . Equally so, the degree of the sum of functions where a leading term has been removed will be lesser than the multidegree the functions to begin with. So the second and third sums of the expanded equation 6 will be less than δ . So, we can focus our attention solely on the monomials of the sum that includes the leading term. This is since if we can prove that the monomials in the sum of the leading terms is less than δ , then the claim of δ minimal property is contradicted. This is where our well studied S-polynomial is used.

Since the sum of all the sums in equation 6 is less than δ , our leading term sum is forced to be less than δ . But this was the exact definition from lemma 3.11. So we can rewrite the sum of leading terms as a combination of S-polynomials. But the consequence of using S-polynomials and lemma 3.11 was precisely that the summands have a lesser multidegree. Then, we have shown that f can be rewritten as a sum of polynomials, all with multidegree less than that minimal δ . Thus, we have contridicted the initial claim. □

This criterion is the primary tool for the algorithm that is going to be constructed and programmed. By having a concrete way of checking if a function is part of a Gröbner bases, an algorithm can be constructed.

4 Algorithms in Macaulay2

4.1 Exploring Macaulay2

After installing the Windows Subsystem for Linux, the Macaulay2 (M2) playground was ready and open. Since the main book [LM21] I was using had an introductory section to the language, the first order of business was to familiarize myself with EMACS and how the syntax of the language worked. This took quite some time as EMACS/Linux is highly unfamiliar to me and the syntax used when interfacing with the M2 console line is different from when one writes a script.

The very first script I wrote, A.7 mirrors this confusion. Even though I am creating a script. Double `*` are being used instead of exponentials because the swedish keyboard has an odd knack for making the powers small. This lead to compiler mismatches and wasted hours. This script is a starting point in how to use the language. Important features such as which ring is being operated upon, which ordering, and how to create a Gröbner basis is touched upon. The next couple of book examples, namely A.8 and A.9 show how to construct more involved rings as well as how to evaluate functions in the ring itself. This eased the understanding of how to use rings in the language.

While helpful, these book examples provided little help in how to construct a script that executes any sort of meaningful code. The book displayed clips from the command line of M2, which is not the proper format for writing algorithms. So even though book example A.7 and A.10 show built in ways to perform division and find the remainder, implementing a division algorithm in one and multiple dimensions seemed like an appropriate start to acclimate myself to the language.

4.2 Division Algorithms

Division algorithms have been of interest for mathematicians for a long time. Euclids algorithm, one of the oldest algorithms created, has applications in for division as well. Basic arithmetic with additions, subtraction, and multiplication can be constrained to $\mathbb{Z}$, the integers, while the introduction of division necessitates an entirely different way of interacting with numbers, $\mathbb{Q}$, the rationals. Division is useful, so a mathematical programming language must be equipped with ways to handle division. Macaulay2 has various ways of dividing, but attempting to extending it to divide polynomials left me with a multitude of problems, highlighting how

complicated algebra can actually be.

4.2.1 Univariate Division Algorithm

Algorithm 1 Univariate Division Algorithm

Require: Functions p, g

Ensure: Quotient q and remainder r

```
1:  $r \leftarrow p$ 
2:  $q \leftarrow 0$ 
3: while  $r \neq 0$  AND  $\deg(r) < \deg(g)$  do
4:    $t \leftarrow \text{LT}(r) / \text{LT}(g)$ 
5:    $q \leftarrow q + t$ 
6:    $r \leftarrow r - gt$ 
7: end while
8: return  $(q, r)$ 
```

The first algorithm I tried writing was [A.1](#). This is basic univariate polynomial division, something that most high schoolers learn. The algorithm is rather simple. Take the leading term of the numerator and divide it by the denominator. Add the quotient to the result and subtract the quotient times the denominator from the numerator. Repeat until the numerator is zero or the degree of the denominator exceeds that of the numerator. None of the ordering issues or really anything at all is difficult in this case. The built in division operator works without any issues. Filled with confidence, the multivariate algorithm was attempted.

4.2.2 Multivariate Division Algorithm

The idea behind the multivariate division algorithm is not much different than the univariate one. Divide a polynomial p by an array of polynomials gs . If the leadterm of p cannot be divided by the leadterm of the first polynomial of gs , move on to the next until one is found. If no leading term of gs can divide the leading term of p , then the leading term is subtracted from p and the algorithm marches to the next polynomial. Seems easy enough, but there is quite a lot of complexity hiding between the lines. The first complexity had to do with the fact that there are more dimensions. Dividing with one dimension is easy enough, but when more are introduced, a way to organize the different dimensions is required. MonomialOrdering is introduced to solve this problem.

Algorithm 2 Multivariate Division Algorithm

Require: Functions $p, g_1, \dots, g_n$

Ensure: Quotients q_s and remainder r

```
1:  $r \leftarrow 0$ 
2:  $f \leftarrow p$ 
3:  $q_s \leftarrow$  empty collection of length  $n$ 
4: while  $f \neq 0$  do
5:    $i \leftarrow 0$ 
6:    $\text{divisionOccurred} \leftarrow \text{false}$ 
7:   while  $i \leq n$  AND  $\text{divisionOccurred} = \text{false}$  do
8:     if  $\text{LM}(f) \mid \text{LM}(g_i) = 0$  then
9:        $t \leftarrow f/g_i$ 
10:       $q_i \leftarrow q_i + t$ 
11:       $f \leftarrow f - g_i t$ 
12:       $\text{divisionOccurred} = \text{true}$ 
13:    else
14:       $i \leftarrow i + 1$ 
15:    end if
16:  end while
17:  if  $\text{divisionOccurred} = \text{false}$  then
18:     $r \leftarrow r + \text{LT}(f)$ 
19:     $f \leftarrow f - \text{LT}(f)$ 
20:  end if
21: end while
22: return  $(q_s, r)$ 
```

Another issue was how to divide the leadmonomials. With the addition of the higher dimension, the regular division was giving fractions of rings. This is an issue as the algorithm wants to remain within the ring itself, not outside of the ring. Foolishly, I decided to try and create a `leadingTermDivision` function that decomposed the polynomials into lists of monomials, coefficients, and exponents. The idea was simple. Macaulay2 generates a list of the exponents taking the ring elements into account. So if the ring as $Q[x, y, z]$, the polynomial $x^2 * z$ would be decomposed into $\{2, 0, 1\}$. By creating these lists and subtracting the exponents from each other, the division has occurred. All that would be left would be to recreate the polynomial using the generators of the ring and raise them to the exponent in the listed order. However, here my lacking understanding of Macaulay2 and more generally ring algebra can be seen. The generators of the ring, and the ring itself are different. In line 63 of [A.2](#), `toList gens ring p` is used. Here, we find the ring of polynomial `p`, find the generators of the ring, and then create a list of it. However, when later applying the exponential list to the generators, the returned polynomial is not an element of the ring that the function caller is using. It simply has the same variables.

Instead of recreating a way to divide monomials or using fractions of rings, there is a division that ensures the result remains in the ring. It does so by using division with remainder. With this realization, the algorithm was ready to be implemented. By first checking if the modulo of the leadmonomial would be 0, we can ensure that the ring division would not produce a remainder. So if the modulo did produce a remainder, the next `gs` was considered. If there was no remainder, essentially the univariate case is established.

One important note here, another reason to use Gröbner bases, is that the order of the polynomials sent in, `gs`, matter significantly. Different ordering of the `gs` list generates different results. A test showing this can be found in [A.2](#). Gröbner bases eliminates this reliance on the order which is desirable.

4.3 The Buchberger Algorithm

Buchbergers Algorithm is the algorithm that finds a complete gröbner basis. The Buchberger criterion that was carefully proven in the previous sections of this thesis is fully leveraged here. In doing so, the algorithm itself is rather easy to implement, all the work behind the algorithm is in proving that the criterion is sufficient to generate the Gröbner basis. However, the criterion does not imply that the algorithm

will terminate in a finite amount of steps. This is important as we want to ensure that a computer can implement the algorithm successfully.

Algorithm 3 Buchberger Algorithm

Require: Functions $P = p_1, \dots, p_n$ that generate I .

Ensure: A Gröbner basis $G = g_1, \dots, g_s$ for I , with $P \subseteq G$

```

1:  $G \leftarrow P$ 
2:  $\hat{G} \leftarrow \{\}$ 
3: while  $G \neq \hat{G}$  do
4:    $\hat{G} \leftarrow G$ 
5:   for every pair  $\{g_i, g_j\}$ ,  $g_i \neq g_j$  in  $\hat{G}$  do
6:      $r \leftarrow \overline{S(g_i, g_j)}^{\hat{G}}$ 
7:     if  $r \neq 0$  then
8:        $G \leftarrow G \cup \{r\}$ 
9:     end if
10:  end for
11: end while
12: return  $G$ 

```

Proof. There are two parts to the proof. First, we must prove that the generated set, G , actually is a Gröbner basis, and then we must prove that the algorithm terminates. We can prove that G is a Gröbner basis by first noting that that $G \subseteq I$ at every point of the algorithm. Since we are solely using functions that generate I , any remainder from division will be on I as well. So by extending G with the remainder, $G \subseteq I$ holds. The fact that the ideal G is a Gröbner basis comes from theorem 3.12. The algorithm terminates when $\overline{S(g_i, g_j)}^{\hat{G}} = 0$ for all pairs which was the condition for the basis to be a Gröbner basis.

The remaining part is to prove that the algorithm terminates. Consider the remainder, r , that is generated from the division of members on G . From lemma 3.2, we can note that the $LT(r)$ is not contained in $\hat{G}$, but is added to G . Thus, we can state that the $\langle LT(\hat{G}) \rangle \subseteq \langle LT(G) \rangle$ since $LT(r)$ is solely in $LT(G)$. Since we are dealing with an ascending chain of ideals in $k[x_1, \dots, x_n]$, the ascending chain condition can be used (see chapter 2.5 theorem 7 in [LM21]). By the ascending chain condition, there exists an $N \geq 1$ such that $I_N = I_{N+1} = I_{N+2} = \dots$. Thus, after N steps of the algorithm, further steps will not add any r , and the algorithm will terminate. $\square$

4.4 Solving Systems of Polynomials

The polynomial solution was by far the most involved algorithm to construct from a Macaulay2 standpoint. At this point, I was feeling much more comfortable with the language, the syntax no longer holding me back, and a more functional approach being taken. I started by following examples from the book [LM21], namely A.5 and A.6.

4.4.1 Book Examples of Polynomial Solutions

To start off, a simply single variable polynomial is considered. The quotient ring is generated, and the monomial basis is found. This is surprisingly easy in Macaulay2, and most of these complex operations are simply done under the hood. One important note, is that the basis function of Macaulay2 generates elements outside the ring, so a nifty "lift" is used to bring them back in. Consider the fact that $\frac{4}{2}$ can be "lifted" from $\mathbb{Q}$ to $\mathbb{Z}$. After this, we find the Q matrix associated with Z , or M_z^Q . While perhaps tedious to do by hand, taking the modulo by the ideal is easy in Macaulay2. Building the coefficients matrix is similarly easy as we simply extract the of the coefficients of the remainder of the modulo with the ideal. All that remains is to raise the matrix to the necessary powers to reconstruct the polynomial that is of interest and one is done.

The second book example, A.6 complicates things slightly by introducing a second variable. However, at this point I realized that a dedicated function to solving M^Q would be necessary. All that needed to be done was to extract the logic used in A.5 and refactored it to its own function. After testing to ensure that they returned the same result, implementing A.6 was easy enough. Solving the M^Q is as easy as sending the ring, ideal, and the monomial of interest for the matrix. Solving the eigenvalues is a piece of cake as well with the built in functionality. Filled to the brim with confidence, the main algorithm was attempted.

4.4.2 Solving a System of Polynomial Equations

This was the final test for everything I had learnt in Macaulay2. Equipped with the function to solve M^Q , I thought it would be easy. Starting off, for a ring $R[x_1, \dots, x_n]$, all the $M_{x_i}^Q$ had to be found. This was easy enough, and finding the eigenvalues thereafter was also something previously done in the book exercises. However, the difficult part came when having to construct the candidate points to

plug back into the initial ideal. Using "fold", which is the easiest way to construct a composite function, and "table", which takes two lists and creates all possible pairs with those two lists. These in conjunction create all possible sets of the eigenlists. I am immensely proud of this and it shows how much I have developed with this language. However, when I evaluate the candidate points over the ideal, to see which give a solution to 0, a slight problem occurs. Since the ring is in R, the computations are not exact, and slight variations cause duplicates. Even adding an epsilon of $1e^{-6}$ in order to check to see if there are duplicate solutions did not help. So, in order to solve this issue, there is an additional "unique solutions check". Here the check is to ensure that the sum of the distance between all the points is lesser than the epsilon.

A Macaulay2 source files

A link to all the source files can be found at the github link [\[Eri26\]](#) under the m2 folder.

A.1 Algorithm121.m2

```
1 univariatePolynomialDivision = (p,g) -> (  
2   q := 0;  
3   r := p;  
4   while (r != 0_R and degree(g) < degree(r)) do (  
5     t := leadTerm(r) / leadTerm(g);  
6     q = q+t;  
7     r = r-t*g;  
8   );  
9   (q,r)  
10  )  
11  
12 runUnivariateTest = () -> (  
13   R = QQ[x];  
14   p := x^6 - 1;  
15   g := x^4 - 1;  
16  
17   (resultQ, resultR) := univariatePolynomialDivision(p,g);  
18  
19   print(resultQ == x^2);  
20   print(resultR == x^2 - 1);  
21 )
```

Listing 1: Macaulay2 script Algorithm121.m2

A.2 Algorithm123.m2

```
1 multivariatePolynomialDivision = (p, gs) -> (  
2   r := 0_(ring p);  
3   f := p;  
4   qs := for i from 0 to #gs-1 list 0_(ring p); --generating  
        dummy variable for result  
5  
6   while f != 0_(ring p) do (  
7     i := 0;
```

```

8      divisionOccured := false;
9      --print "Outer loop";
10
11     while (i < #gs and divisionOccured == false) do (
12         --print "Inner loop";
13         lmf := leadMonomial f;
14         lmg := leadMonomial (gs#i);
15
16         if (lmf % lmg == 0) then (
17             monomialQuotient := lmf // lmg;
18
19             coefficientQuotient := leadCoefficient f /
20                 leadCoefficient gs#i;
21
22             t := coefficientQuotient * monomialQuotient;
23
24             qs = replace(i, qs#i + t, qs);
25             f = f - t*gs#i;
26
27             divisionOccured = true;
28             --print "we were divisible";
29         )
30         else (
31             i = i+1;
32
33             --print "not divisible, function further along";
34         )
35     );
36
37     if divisionOccured == false then (
38         r = r + leadTerm(f);
39         f = f - leadTerm(f);
40         --print "division did not occur, subtracting leading
41             term and continuing";
42     );
43     return (qs, r)
44 )
45
46 -- This below code is useless, it was an attempt to get around the
47    fact that i couldnt divide with the leadTerm.

```



```

46  -- However, I needed to seperate out the divisions as well as use
    "/" instead of "/"
47  leadingTermDevision = (p,g) -> (
48      lmP := leadMonomial(p);
49      lmG := leadMonomial(g);
50      lcP := leadCoefficient(p);
51      lcG := leadCoefficient(g);
52      exponentsP := flatten exponents lmP;
53      exponentsG := flatten exponents lmG;
54
55      divisible := all(#exponentsP, i -> exponentsP#i >=
        exponentsG#i); --checking that all exponents are greater or
        equal
56      result := 0_(ring p);
57
58      if divisible then(
59          coefficientResult := lcP / lcG;
60
61          exponentsQ := for i from 0 to (#exponentsP-1) list
            (exponentsP#i - exponentsG#i); --doing the division
62
63          ringVariables := toList gens ring p;
64
65          Q := product(#ringVariables, i -> ringVariables#i ^
            (exponentsQ#i)); --generating the the polynomial
66
67          result = coefficientResult * Q;
68      );
69
70      return (divisible, result);
71  )
72
73  runMultimultivariateTest = () -> (
74      R = QQ[x,y,MonomialOrder=>Lex];
75      p := -x^2*y-x^2+x*y+y^3;
76      g1 := x*y-x+y-1;
77      g2 := x+y+1;
78
79      (resultG1, resultRemainder1) :=
        multivariatePolynomialDivision(p,{g1,g2});
80      (resultG2, resultRemainder2) :=
        multivariatePolynomialDivision(p,{g2,g1});

```

```

81
82     print "-----{g1,g2}-----";
83     print resultG1;
84     print resultRemainder1;
85     print (resultG1#0 == -x+4);
86     print (resultG1#1 == -2*x+5);
87     print (resultRemainder1 == y^3-9*y-1);
88
89     print "-----{g2,g1}-----";
90     print resultG2;
91     print resultRemainder2;
92     print (resultG2#1 == 0);
93     print (resultG2#0 == -x*y-x+y^2+3*y+1);
94     print (resultRemainder2 == -4*y^2-4*y-1);
95 )

```

Listing 2: Macaulay2 script algorithm123.m2

A.3 Algorithm124.m2

```

1  load "algorithm123.m2";
2
3  buchberger = ps -> (
4      g := ps;
5      gHat := {0_(ring ps#0)};
6
7      while g != gHat do (
8          gHat = g;
9
10         -- generating all the possible pairs by looping through i
11         and then from j -> end
12         polynomialPairs := flatten for i from 0 to #gHat-1 list
13             for j from i+1 to #gHat-1 list (gHat#i, gHat#j);
14
15         i := 0;
16         while i < #polynomialPairs do (
17             p := (polynomialPairs#i)#0;
18             q := (polynomialPairs#i)#1;
19
20             xDelta := lcm(leadMonomial(p), leadMonomial(q));

```

```

20         s := xDelta // leadTerm(p) * p - xDelta // leadTerm(q)
21             * q;
22
23         (qs, r) = multivariatePolynomialDivision(s, gHat);
24
25         if r != 0_(ring ps#0) then (
26             g = append(gHat, r);
27         );
28
29         i = i+1;
30     );
31     print g;
32     return g;
33 )
34
35 testAlgorithm = () -> (
36     R = QQ[x,y, MonomialOrder => GLex];
37
38     p1 := x^2 + y;
39     p2 := x^4 + 2*x^2*y + y^2 + 3;
40
41     myBasis := gb ideal (buchberger({p1, p2}));
42     correctBasis := gb ideal(p1, p2);
43     areTheyEqual := myBasis == correctBasis;
44
45     print "My basis:";
46     print myBasis;
47
48     print "Built-in basis:";
49     print gens correctBasis;
50
51     print "Are they equal?";
52     print areTheyEqual;
53 )
54
55 --again, this is not needed and M2 implements LCM.
56 findXDelta = (p,q) -> (
57     alpha := flatten exponents leadMonomial(p);
58     gamma := flatten exponents leadMonomial(q);
59

```

```

60     delta := for i from 0 to (#alpha-1) list
        (max(alpha#i,gamma#i)); --getting the max of each variables
        exponent
61
62     ringVariables := toList gens ring p;
63     xDelta := product(#ringVariables, i -> ringVariables#i ^
        (delta#i)); -- reconstructing the variable
64
65     return xDelta
66 )
67
68 testGamma = () -> (
69     R := RR[x,y, MonomialOrder => GLex];
70     p := x^3*y^2-x^2*y^3+x;
71     q := 3*x^4*y +y^2;
72
73     xDelta := findXDelta(p,q);
74     xDeltaShouldBe := x^4*y^2;
75
76     print "-----gamma results-----";
77     print xDelta;
78     print xDeltaShouldBe;
79     print("the result is " | toString(xDelta == xDeltaShouldBe))
80 )

```

Listing 3: Macaulay2 script algorithm124.m2

A.4 Algorithm161.m2

```

1 solveSystemOfPolynomialEquations = (theRing, theIdeal) -> (
2     varsToLoopOver := gens theRing;
3
4     -- get the list of eigenvalues
5     eigenList := for var in varsToLoopOver list (
6         qMatrix := solveQMatrix(theRing, theIdeal, var);
7
8         --ensure that the eigenvector can be solved and save it
            to the list
9         toList eigenvalues(sub(qMatrix,CC))
10    );
11

```

```

12  -- use fold to go through all elements in the eigenList
    outwards
13  candidatePoints := fold((L1, L2) -> (
14      --use table to create a table of all the pairs and
        flatten down
15      flatten table(L1, L2, (a, b) -> (
16          --if we are past first instance, we append
            instead of create a list
17          if instance(a, List) then a | {b} else {a, b}
18          )
19      )
20      ),
21      eigenList);
22
23  epsilon := 1e-6;
24  functionList := flatten entries gens theIdeal;
25
26  validSolutions := select(candidatePoints, pt -> (
27      -- ensure that x_n maps to the the nth point
        subMap = apply(#varsToLoopOver, i -> varsToLoopOver#i
28          => pt#i);
29      -- sub in the values
        all(functionList, p -> abs(sub(p, subMap)) < epsilon)
30      ));
31
32
33  -- something is wrong with my solveQMatrix which results in
    cluttered returns.
34  -- so for now just to check if it works, we remove the
    duplicates
35  uniqueSolutions := {};
36  scan(validSolutions, v -> (
37      if not any(uniqueSolutions, u -> (
38          dist := sqrt(sum(0..#v-1, i -> abs(v#i -
            u#i)^2));
39          dist < epsilon
40          )) then uniqueSolutions =
            append(uniqueSolutions, v);
41      ));
42
43  uniqueSolutions
44  )
45

```

```

46
47 testAlgorithm2D = () -> (
48     R := CC[x, y];
49     I := ideal(x^2 + y^2 - 5, x*y - 2);
50
51     myResults := solveSystemOfPolynomialEquations(R, I);
52     solution := {{-2, -1}, {-1, -2}, {2, 1}, {1, 2}};
53
54     print ("The 2D test output: " | toString myResults);
55
56     print ("Verified solution: " | toString solution);
57 )
58
59 testAlgorithm4D = () -> (
60     R4 := CC[x1, x2, x3, x4];
61
62     p1 := x1^2 + x2^2 - 2;
63     p2 := x1 - x2;
64     p3 := x3^2 + x4^2 - 13;
65     p4 := x3*x4 - 6;
66     I4 := ideal(p1, p2, p3, p4);
67
68     results4D := solveSystemOfPolynomialEquations(R4, I4);
69
70     if #results4D == 8 then (
71         print "SUCCESS: Found all 8 points in 4D space.";
72     ) else (
73         print ("FAILURE: Expected 8 points, but found " |
74             #results4D);
75     );
76
77 solveQMatrix = (theRing, theIdeal, thePolynomial) ->(
78     q := theRing / theIdeal;
79
80     basisList := flatten entries basis q;
81     polyBasis := for b in basisList list lift(b, theRing); --basis
82         to R
83
84     rows := for b in polyBasis list (
85         currentPoly := thePolynomial * b;
86         r := currentPoly % theIdeal;

```

```

86
87     coeffsMatrix := last coefficients(r, Monomials =>
88         polyBasis);
89     flatten entries coeffsMatrix
90 );
91
92     return transpose matrix rows;
93 )
94
95 print "-----2D CASE-----";
96 testAlgorithm2D();
97 print "-----4D CASE-----";
98 testAlgorithm4D();
99 print "-----DONE-----";

```

Listing 4: Macaulay2 script algorithm161.m2

A.5 BookExample1612.m2

```

1  runExample = () -> (
2      R = RR[z];
3
4      g := z^3-3*z^2+2*z;
5
6      I = ideal g;
7
8      q := R / I;
9
10     basisList := flatten entries basis q;
11     polyBasis := for b in basisList list lift(b, R); -- ensuring
12                  that the actualy polynomials exists in R and not only in q
13
14     rows := for b in basisList list (
15         currentPoly = z *lift(b,R);
16
17         r := currentPoly % I;
18
19         --taking #1 here since coefficients returns a bundle of
20         want the coefficients, no the monomials
21         coeffsMatrix := (coefficients(r, Monomials =>
22             polyBasis))#1;
23         flatten entries coeffsMatrix

```

```

21         );
22
23     zmatrix := matrix rows;
24
25     finalZMatrix := zmatrix^2+zmatrix+id_(target zmatrix);
26     return finalZMatrix;
27 )
28
29
30 runExampleWithFunction = () -> (
31     R = RR[z];
32
33     g := z^3-3*z^2+2*z;
34
35     I = ideal g;
36
37     load "algorithm161.m2";
38
39     zmatrix := solveQMatrix(R, I, z);
40
41     finalZMatrix := zmatrix^2+zmatrix+id_(target zmatrix);
42     return finalZMatrix;
43 )
44
45 testFunction = () -> (
46     preBuiltMatrix := runExample();
47     matrixFromFunction := runExampleWithFunction();
48
49     print "Matrix from the example";
50     print preBuiltMatrix;
51
52     print "Matrix using refactored function";
53     print matrixFromFunction;
54 )

```

Listing 5: Macaulay2 script Example1612.m2

A.6 BookExample1613.m2

```

1 runExample = () -> (
2     load "algorithm161.m2";
3

```



```

4      R = RR[x,y];
5      f := (y^2+x-3, x^2-4*x*y-y^2-10*y+15);
6      I := ideal f;
7
8      regMatrix := solveQMatrix(R,I,1);
9      xMatrix := solveQMatrix(R,I,x);
10     xyMatrix := solveQMatrix(R,I,x*y);
11     yMatrix := solveQMatrix(R,I,y);
12
13     --only care about the x,y individual ones
14     xEigens := (eigenvectors sub(xMatrix,CC))#0;
15     yEigens := (eigenvectors sub(yMatrix,CC))#0;
16
17     varietyList := for i from 0 to #xEigens-1 list {xEigens#i,
18               yEigens#i};
19     print varietyList;
20 )

```

Listing 6: Macaulay2 script Example1613.m2

A.7 BookExample2.1.1.m2

```

1  R = QQ[x_1..x_4, MonomialOrder => Lex]
2
3  p1 = x_1-x_2**2+x_3
4
5  p2 = x_1**2+x_2*x_3**2+x_4
6
7  I = ideal(p1,p2)
8
9  G = gens gb I
10
11  polyToDivide = x_1**2+x_2**2+x_3**2+x_4**2-1
12
13  rem = polyToDivide%I
14
15  print("the rem is: ")
16  print(rem)

```

Listing 7: Macaulay2 script BookExample2.1.1.m2

A.8 BookExample2.2.m2

```
1 R = QQ[x_1,x_2,x_3]
2
3 p = (1/2)+x_1^2*x_2*x_3^4+(6/5)*x_1*x_3+2
4
5 -- output is a polynomial in R
```

Listing 8: Macaulay2 script BookExample2.2.m2

A.9 BookExample222.m2

```
1 R = frac(QQ[a]/ideal(a^2-2))[x_1,x_2,x_3]
2
3 substitute(a^2, R)
4 -- output is that a^2 = 2
5
6 p = x_1 + x_2 + a*x_3
```

Listing 9: Macaulay2 script BookExample222.m2

A.10 BookExample231.m2

```
1 R = QQ[z]
2
3 p = z^4+3*z^3-3*z^2+3*z-4
4
5 pc = -2*z^4+3*z^3-6*z^2+5*z-4
6
7 g = z^2+1
8
9 (q,r)= quotientRemainder(p,g)
10
11 (qc, rc) = quotientRemainder(pc,g)
```

Listing 10: Macaulay2 script BookExample231.m2

A.11 BookExample232.m2

```

1 R=QQ[c_1,c_2,c_3,c_4, MonomialOrder=>Lex]
2 M = matrix{{1,-2,0,-1},{4,0,7,3}, {7,-6,7,0}}
3 C = matrix{{c_1},{c_2},{c_3},{c_4}}
4 I = ideal(M*C)
5 GI = value toString gens gb I
6 (Ce,Me) = value toString coefficients(GI,
    Monomials=>{c_1,c_2,c_3,c_4})
7
8 Mr = value toString transpose Me
9
10 --powerful way to solve the row reduced form
11 inverse(substitute(submatrix(Mr, {0,1}),QQ))*Mr

```

Listing 11: Macaulay2 script BookExample232.m2

A.12 DocumentationConditionals.m2

```

1 isEven = i -> i % 2 == 0
2
3 oddOrEven = i -> (
4     if isEven i then "even"
5     else "odd")

```

Listing 12: Macaulay2 script documentationConditionals.m2

References

- [DAC10] Donal B O'Shea David A Cox, John Little. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer Publishing Company, third edition, November 2010.
- [DF03] David S. Dummit and Richard M. Foote. *Abstract Algebra*. Wiley, Hoboken, NJ, USA, 3 edition, 2003.
- [Eri26] Christian Eriksson. M2 algorithms. Available at <https://github.com/Skalmanen/MM6010>, 2026.
- [Gal25] Jean H. Gallier. Basics of affine geometry. In Jean H. Gallier, editor, *Geometric Methods and Applications for Computer Science and Engineering*, pages 7–63. Springer, 2025. Chapter 2 from online course materials. URL: <https://www.cis.upenn.edu/~cis6100/geombchap2.pdf>.
- [LM21] Antonio Tornambè Laura Menini, Corrado Possieri. *Algebraic Geometry for Robotics and Control Theory*. WORLD SCIENTIFIC (EUROPE), 2021. [arXiv:https://www.worldscientific.com/doi/pdf/10.1142/q0308](https://www.worldscientific.com/doi/pdf/10.1142/q0308).
- [Wei70] Louis M. Weiner. *Introduction to Modern Algebra*. Harcourt, Brace & World, New York, NY, USA, 1970.